

Web Tooling

Set of tools developed for ITB Insight's website analytics. Given the multi-division nature of ITB Insight, create one **events** table, one row per-section, categorized by **division_tag** who'd want their own numbers. Later, stakeholders of each division's account will be given a 'stakeholder' role, which lets them see the whole of the metrics, not just their own coded ones. The tags are just there so I don't regret anything if plans change in the future.

Codename for Each Body / Division of Interest

Competition : CPT

Event : EV

Sponsorship : SP

Marketing : MKT

Creative Branding : CB

Misc : MISC

Since some of the pages are either incomplete, or there will be new pages/components in the future, please make a template-ish of features like the drop-off points mechanism and the clicks/impressions so that it may be applied into any page in the future, and make .md documents explaining in-brief how the feature works. SCOPE EVERYTHING JUST FOR THE MVP FOR NOW, and document what needs to be fixed/complimented in the future for the full implementation

Tools

Generic event logger (/api/track)

Single end-point `logEvent('sponsor_click', {sponsorId, tier, division: 'SPON'})`. Build once, use everywhere.

Sponsor Dashboard

- Clicks per sponsor logo/banner

- Impressions (did the sponsor's section scroll into view [IntersectionObserver])
- Click-through rate by tier (if you have tiers)

Sponsors often want proof of value, so log **impressions AND clicks**.

Funnel Dashboard

- Landing → Registration started → Registration completed (per-competition/event)
- Drop-off points – where do people abandon the form?
- QR check-in scan rate vs. registered count (shows actual turnout vs.sign-ups)

Engagement Dashboard (for 3D flythrough / general UX)

- Scroll depth (did people even get through the
- Time to interactive / load performance – 3D objects are heavy. < Please create just a simple Three.js 3D cube for the 'drone' component.
- Device / browser breakdown. Critical since WebGL performance varies wildly on lower-end phones.

Advice

- **Batch client-side events** (queue and flush every few seconds or on page hide) rather than firing a request per click – with sponsor logo hovers and scroll tracking this adds up fast and you don't want to hammer your API.
- **Don't log PII in the metadata JSONB** casually – if `user_id` is null for anonymous visitors, keep it that way; don't backfill emails into event metadata just because it's convenient later.
- **Rate-limit the tracking endpoint itself** – you already have WAF/rate-limiting planned for security hardening, make sure `/api/track` is covered too, since it's a public write endpoint and an easy target for junk data.

- **Sample if traffic gets heavy** – for high-volume events like scroll tracking, log 1-in-N rather than every single scroll tick. Full fidelity on clicks/conversions, sampled on passive engagement signals.
- **Build the dashboard queries as materialized views or scheduled aggregates**, not live COUNT() queries on raw events – event day traffic spikes and you don't want your admin dashboard queries competing with the site itself for DB resources.
- For now, just create mock / hard-coded for the data.

Schema

```
-- Core event log
CREATE TABLE events (
    id BIGSERIAL PRIMARY KEY,
    session_id UUID NOT NULL,
    user_id UUID, -- nullable, only set if logged in (e.g. admin
previewing)
    event_type TEXT NOT NULL,          -- 'sponsor_click',
'reg_started', 'reg_completed', 'qr_checkin', 'scroll_depth',
'page_view'
    division TEXT NOT NULL,           -- 'MK', 'CP', 'EV', 'CB', 'SPON'
    page_path TEXT NOT NULL,
    metadata JSONB DEFAULT '{}',      -- flexible per-event-type
payload
    user_agent TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_events_type_division ON events (event_type,
division);
CREATE INDEX idx_events_created_at ON events (created_at);
CREATE INDEX idx_events_session ON events (session_id);
-- GIN index only if you'll query inside metadata often (e.g.
sponsorId)
```

```

CREATE INDEX idx_events_metadata_gin ON events USING GIN (metadata);

-- Registrations (structured, not just events – you need this as
source of truth)
CREATE TABLE registrations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id UUID NOT NULL,
    competition_id TEXT,          -- nullable if it's a general
event, not competition
    division TEXT NOT NULL,
    status TEXT NOT NULL DEFAULT 'started', -- 'started', 'completed',
'checked_in'
    started_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    completed_at TIMESTAMPTZ,
    checked_in_at TIMESTAMPTZ
);

-- Sponsors (source of truth for the dashboard join)
CREATE TABLE sponsors (
    id TEXT PRIMARY KEY,
    name TEXT NOT NULL,
    tier TEXT NOT NULL -- 'platinum', 'gold', 'silver'
);

-- Materialized view for the sponsor dashboard (refresh on a schedule,
not live)
CREATE MATERIALIZED VIEW sponsor_stats AS
SELECT
    s.id AS sponsor_id,
    s.name,
    s.tier,
    COUNT(*) FILTER (WHERE e.event_type = 'sponsor_impression') AS
impressions,
    COUNT(*) FILTER (WHERE e.event_type = 'sponsor_click') AS clicks,
    ROUND(
        COUNT(*) FILTER (WHERE e.event_type =
'sponsor_click')::numeric

```

```

        / NULLIF(COUNT(*) FILTER (WHERE e.event_type =
'sponsor_impression'), 0) * 100, 2
    ) AS ctr_percent
FROM sponsors s
LEFT JOIN events e ON e.metadata->>'sponsorId' = s.id
GROUP BY s.id, s.name, s.tier;

-- Refresh via cron / pg_cron:
-- REFRESH MATERIALIZED VIEW CONCURRENTLY sponsor_stats;

```

Example dashboard queries

Registration funnel drop-off (per competition):

```

SELECT
    competition_id,
    COUNT(*) FILTER (WHERE status IN
('started', 'completed', 'checked_in')) AS started,
    COUNT(*) FILTER (WHERE status IN ('completed', 'checked_in')) AS
completed,
    COUNT(*) FILTER (WHERE status = 'checked_in') AS checked_in,
    ROUND(
        COUNT(*) FILTER (WHERE status IN
('completed', 'checked_in'))::numeric
        / NULLIF(COUNT(*) FILTER (WHERE status IN
('started', 'completed', 'checked_in')), 0) * 100, 1
    ) AS completion_rate_pct
FROM registrations
GROUP BY competition_id
ORDER BY started DESC;

```

Scroll depth bail-out on the 3D flythrough (by device class):

```

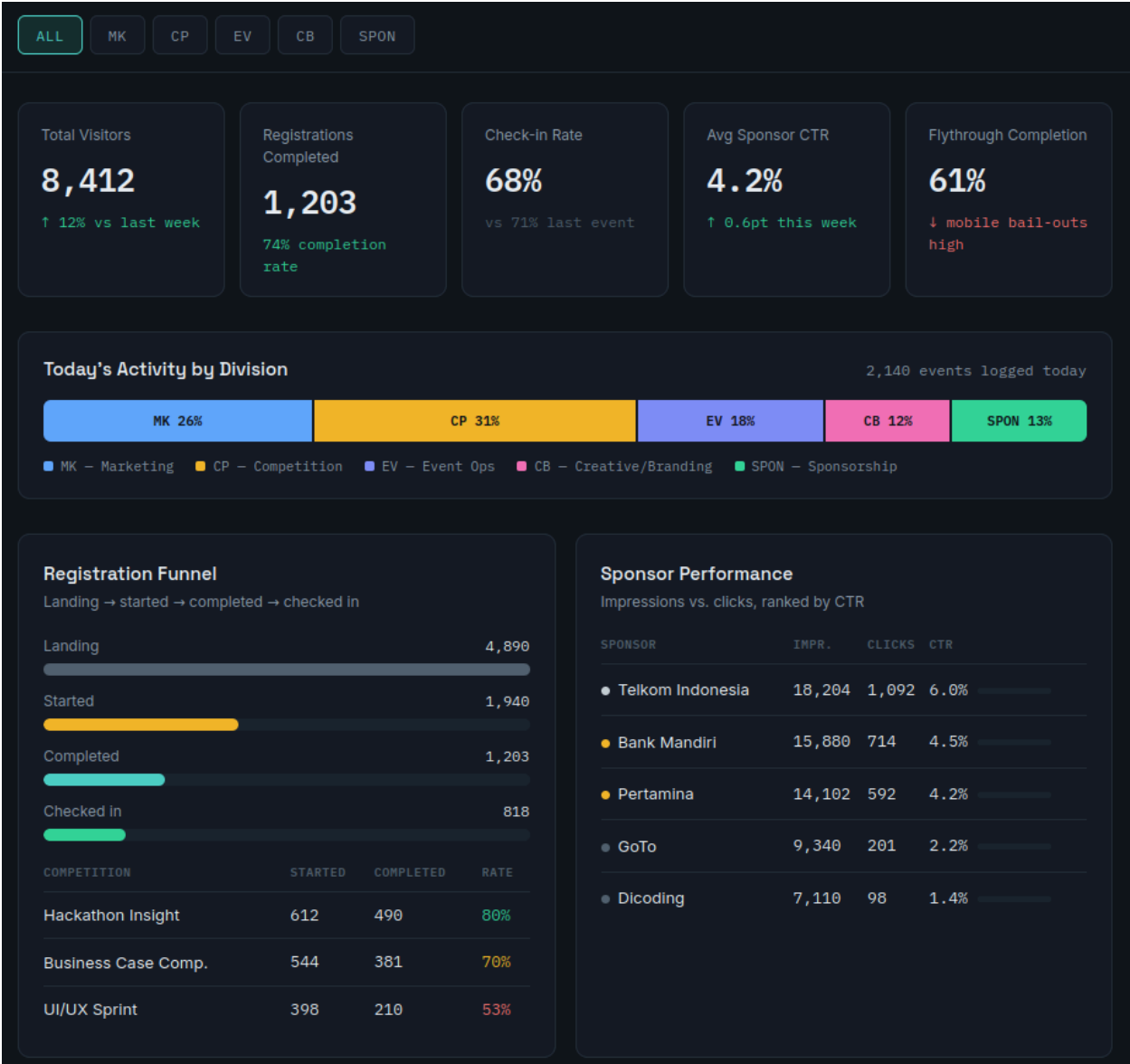
SELECT
    metadata->>'deviceClass' AS device_class,
    ROUND(AVG((metadata->>'scrollDepthPct')::numeric), 1) AS
avg_scroll_depth,

```

```
        COUNT(*) FILTER (WHERE (metadata->>'scrollDepthPct')::numeric <
20) AS early_bailouts,
        COUNT(*) AS total_sessions
FROM events
WHERE event_type = 'scroll_depth' AND page_path = '/flythrough'
GROUP BY device_class;
```

Daily traffic + top pages (general admin overview):

```
SELECT
    DATE_TRUNC('day', created_at) AS day,
    page_path,
    COUNT(*) AS views
FROM events
WHERE event_type = 'page_view'
GROUP BY day, page_path
ORDER BY day DESC, views DESC;
```





● Dashboard

● Users

● Events

● Competition

● Content

● Partners

● Analytics

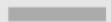
● Settings

Search bar with magnifying glass icon and two filter buttons

●	
●	
●	
●	
●	
●	

Logo ITB Insight

Dashboard



● Dashboard

● Users

● Events

● Competition

● Content

● Analytics

● Settings

